[version_RG] A stepwise procedure for inferring gene regulatory networks and pseudotime-based trajectories from single-cell RNA-sequencing data

Riveras E.[1†], Vergara M.[1†], Moreno-Ramirez S.[2], Gutiérrez RA[1*]

[1]Facultad de Ciencias Biológicas, Pontificia Universidad Católica de Chile. Millennium Institute for Inte

2026

Contents

Abstract **3**

Introduction **4**
About this document . . . 4
Minimum computing requirements . . . 5
Recommended: persistent terminal sessions with tmux or screen . . . 5

1 Part 1 - Environment and raw data **7**
1.1 Clone the repository . . . 7
1.2 Container startup (Docker or Singularity) . . . 7
1.2.1 Which container runtime should I use? . . . 8
1.3 Raw data download and Cell Ranger processing . . . 9

2 Part 2 - Single-cell analysis in R **13**
2.1 Initialization . . . 13
2.2 Sections 1-4 - Preprocessing: QC, filtering, merging and Harmony . . . 14
2.3 Sections 5-6 - Clustering resolution and final clustering . . . 16
2.4 Sections 7-8 - Annotation and gene expression . . . 17
2.5 Section 9 - Export the annotated object . . . 17

3 Part 3 - Pseudobulk differential expression and GO **18**
3.1 Sections 10-16 - Pseudobulk, DESeq2, volcano, tables, GO and heatmap . . . 18
3.2 Section 17 - Gene-gene co-expression network (hdWGCNA) and TF network . . . 19

4 Part 4 - Pseudotime trajectory analysis (Python) **20**
4.1 Section 18 - Setup . . . 20
4.2 Sections 19-23 - Trajectory inference and gene trends . . . 20
4.3 Section 24 - Final export . . . 22

Supplementary methods **25**
Supplementary Section S1 - Annotated clustree . . . 25
Supplementary Section S2 - Cell-type grouping . . . 25
Supplementary Section S3 - Subcluster inspection and manual curation . . . 26
Supplementary Section S4 - Manual root override . . . 29
Supplementary Section S5 - Running the workflow from a parameter file . . . 29

Supplementary Table — Software dependencies and versions **31**
Interpreters and command-line tools . . . 31
Installing tmux or screen . . . 31
Installing a container runtime (Docker or Singularity/Apptainer) . . . 31
Troubleshooting Singularity/Apptainer . . . 32
R packages (421) . . . 32
Python packages (207) . . . 36

Abstract

A stepwise framework for inferring gene regulatory networks and pseudotime-based trajectories from single-cell RNA-sequencing data

Riveras E.[1], Vergara M.[1], Moreno-Ramirez S.[2], Gutiérrez R.A.[1]

[1]Departamento de Genética Molecular y Microbiología, Facultad de Ciencias Biológicas, Pontificia Universidad Católica de Chile, Santiago, Chile. Millennium Institute for Integrative Biology, Millennium Institute for Center for Genome Regulation. [2]Sainsbury Laboratory, University of Cambridge, Bateman Street, Cambridge CB2 1LR, UK.

Plant development is a dynamic process in which new organs are continuously formed throughout an individual's lifetime. This continuous production of new organs enables plants to increase in size and generate tissues with diverse physiological and functional capabilities. Single-cell RNA sequencing (scRNA-seq) has emerged as a powerful approach for characterizing cell-type-specific transcriptomes at high resolution across tissues and species. However, the bioinformatics analyses required to extract biological information from scRNA-seq datasets can be challenging for researchers without formal training in bioinformatics. Here, we provide a step-by-step workflow for inferring gene regulatory networks and pseudotime-based developmental trajectories from scRNA-seq data. To facilitate the implementation and reproducibility of these analyses, we provide a Docker image and a GitHub repository (github.com/laboratoriorgit-lang/SingleCell) containing all scripts, helper functions, reference files, container configuration, and environment YAML files (plain-text configuration files that store settings in a human-readable format). The workflow was applied to publicly available scRNA-seq data from *Arabidopsis thaliana* seedlings and enables the identification of differentially expressed genes (DEGs) across cell types, gene regulatory networks, and developmental trajectories associated with ATML1-driven epidermal cell differentiation in cotyledons. Although this guide focuses on scRNA-seq analysis in *A. thaliana*, the workflow can be adapted to other organisms using appropriate reference genomes, genome annotations, and experimental designs.

Introduction

Plant organs are not laid down once during embryogenesis but are produced continuously across the plant's life, from meristems that generate new roots, leaves and flowers on demand. This ongoing organogenesis depends on precisely coordinated, cell-type-specific gene expression programs, as neighboring cells within the same tissue can follow markedly different transcriptional trajectories depending on their developmental origin, position, and stage of differentiation. Bulk RNA-seq, which averages expression across every cell in a sample, is poorly suited to resolving this heterogeneity: rare cell types are diluted out, and transitions between cell states are collapsed into a single average that may not represent any individual cell.

Single-cell RNA sequencing (scRNA-seq) addresses this limitation directly by profiling transcriptomes one cell at a time, making it possible to resolve cell-type identity, cell-state transitions, and developmental trajectories within a tissue at a resolution bulk approaches cannot reach. Once cells are clustered and annotated, the same data can support several complementary analyses: differential expression between cell types or genotypes, inference of gene regulatory networks from co-expression patterns, and reconstruction of pseudotime trajectories that order cells along a developmental or differentiation axis. In practice, however, each of these steps involves a distinct set of tools, file formats and statistical choices, and combining them into a coherent, reproducible pipeline requires computational expertise that is not always available in plant biology labs, which can be a real barrier to adopting scRNA-seq despite its potential.

This guide provides a complete, reproducible workflow that carries scRNA-seq data from raw sequencing reads through all of these downstream analyses. Starting from FASTQ files, Cell Ranger – a 10x Genomics program that aligns reads to a reference genome and counts, for every gene, how many transcripts were detected in each individual cell – is used to generate a per-cell count matrix (Part 1): this matrix (genes x cells) is the starting point for everything else in this workflow. These matrices are then loaded into R, where cells are filtered, doublets are removed, samples are merged and integrated with Harmony, and clusters are annotated to cell types using literature-curated marker genes (Part 2). From the annotated object, the pipeline builds pseudobulk profiles per cell type to identify differentially expressed genes between genotypes, runs GO enrichment on those gene sets, and infers a transcription-factor co-expression network to highlight candidate regulators (Part 3). Finally, the workflow switches to Python (scanpy, Palantir, scFates) to reconstruct pseudotime trajectories within a selected cell population, order cells along that trajectory, and identify genes whose expression trends define it (Part 4). Every step is packaged as a single container image, so the same software environment — R, Python, and every package version — is available to anyone who pulls the image, removing a major source of "works on my machine" irreproducibility.

To illustrate the complete workflow end to end, this document applies it to a public scRNA-seq dataset comparing wild-type and *pifq* mutant *Arabidopsis thaliana* seedlings, from raw Cell Ranger processing through cell-type annotation, differential expression, regulatory network inference, and a pseudotime trajectory of ATML1-driven epidermal differentiation in cotyledons. Although the example organism is *A. thaliana*, none of the workflow logic is specific to it: the same steps apply to other species by substituting the reference genome, annotation, and marker gene set used for cell-type calling.

About this document

Condensed version (version_RG). This variant of the guide runs the same analysis as the step-by-step guide, but each chapter is a single call to a high-level function (`run_preprocessing()`, `run_clustering()`, `run_annotation()`, `run_pseudobulk_de()`, `run_tf_network()` in R and `run_pseudotime()` in Python). Each function runs several consecutive sections, with the same calls and the same saved PDFs and objects as the step-by-step guide. The functions are defined in `workflow/ScRNA_Pipeline_Wrappers.R` and `workflow/ScRNA_Pseudotime_Wrappers.py`.

Important: this guide uses a subset of the published data. The study by Han et al. (2023) contains many more samples than the two analyzed here. This guide uses only two of them (`WT` and `pifq`) so that the whole workflow can be run in reasonable time on a single machine. It is **not** the same dataset as in the paper, so the clusters, differentially expressed genes, networks and trajectories obtained here are not expected to reproduce the paper's figures or numbers. The goal is to demonstrate the workflow, not to replicate the

published results.

This methods-paper R Markdown documents the full WT vs pifq workflow: two samples, `WT` and `pifq`, processed and integrated together, then carried through three chapters — single-cell preprocessing, clustering and annotation (Part 2); pseudobulk differential expression, GO and the co-expression network (Part 3); and pseudotime trajectory analysis (Part 4). The code blocks are shown exactly as in the methods paper (not executed inline); the figures below are the **real outputs** of a fresh run of `workflow/step1_singlecell.R`, `workflow/step2_degs.R` and `workflow/step3_pseudotime.py`, written to a separate `results_rerun/` directory using the bibliography annotation (`celltype`).

The active annotation path is bibliography-based (`biblio_marks.txt -> celltype_curated`). Reference-transfer annotation is not part of this draft.

Minimum computing requirements

The pipeline has two stages with different requirements:

Cell Ranger (FASTQ → count matrices): see the official 10x Genomics system requirements at https://www.10xgenomics.com/support/software/cell-ranger/downloads/cr-system-requirements. Cell Ranger is CPU- and memory-intensive; 8+ cores and 64+ GB RAM are typical.

R/Python analysis (Section 2 onwards): lighter requirements, suitable for a personal computer with ~10,000 cells.

Component	Minimum	Recommended
CPU cores	4	8 or more
RAM	8 GB	32 GB for merged multi-sample objects
Disk space	80 GB free	100 GB free
Container runtime	Docker or Singularity/Apptainer	Singularity/Apptainer on a cluster; Docker on a personal computer

These requirements were estimated from test runs of the two-sample WT vs pifq dataset. The RAM requirement scales with total cell count (~4 GB per 10k cells for a Seurat object).

This workflow is packaged as a single public image (`psblab/scrnaseq:latest`, available at hub.docker.com/r/psblab/scrnaseq); the `docker-compose` YAML (a configuration file that defines how to build and run the container) used to build and run it is kept in the repository. Any container runtime able to run that image works; the setup below covers both Docker and Singularity/Apptainer (the latter common on HPC clusters, where Docker is usually not permitted).

Small pilot datasets can be run on a laptop or desktop if memory is sufficient. For full experiments, repeated clustering sweeps, Cell Ranger processing from FASTQ, or large merged Seurat objects, a server or external cluster is strongly recommended. Cell Ranger is CPU- and memory-intensive and is not installed inside the downstream `psblab/scrnaseq:latest` Docker image.

Recommended: persistent terminal sessions with tmux or screen

Several steps in this pipeline run for a long time without any interaction — Cell Ranger takes 30-60 minutes per sample, and the R/Python clustering and pseudotime steps can run for hours on large merged objects. If these are launched directly in an SSH session and the connection drops (network hiccup, laptop sleep, closed terminal), the process is killed along with it.

Running everything inside a `tmux` or `screen` session avoids this: the session keeps running on the server after you disconnect, and can be reattached later — from the same machine or a different one — to check progress

or pick up where things were left off. This is strongly recommended any time this pipeline runs on a remote server or HPC cluster, and is still useful on a local machine to keep a session alive while doing other work.

Either tool works; `tmux` is used in the examples below.

Bash

```bash
tmux new -s scrnaseq     # start a new named session
# ... run Cell Ranger, Docker/Singularity, R, etc. inside it ...
# detach without stopping anything: Ctrl-b, then d

tmux attach -t scrnaseq  # reattach later
tmux ls                  # list running sessions
```

This pipeline's R/Python environment ships as a single public container image (`psblab/scrnaseq:latest`, Docker or Singularity/Apptainer). It mounts the cloned repository as `/workspace`, so results are written back into the repo. Always pull the image rather than rebuilding it from the `Dockerfile`, so everyone gets the exact tested package versions (details and commands in "Container startup", Part 1).

Once inside a container (Docker or Singularity) shell, stay inside it for the rest of the work. The R and Python packages this pipeline needs (Seurat, scanpy, Palantir, scFates, etc.) are installed only inside the `psblab/scrnaseq:latest` image — they are not available on the bare host. Leaving the container shell (e.g. `exit`, or the shell dying) means the next command expecting those packages will fail. Pair this with tmux/screen: open a session, enter the container from inside it, and do the rest of the R/Python work (Parts 2-4) without leaving either until finished.

Installation instructions for tmux, screen, Docker and Singularity/Apptainer are in the Supplementary section at the end of this document.

1 Part 1 - Environment and raw data

1.1 Clone the repository

Choose or create a working directory, then clone the pipeline repository into it:

Bash

```bash
mkdir -p ~/my-analysis
cd ~/my-analysis
git clone https://github.com/laboratoriorgit-lang/SingleCell.git
cd SingleCell
```

All paths referenced later in this document (`workflow/`) are relative to this cloned repository. Its layout is:

```
SingleCell/
|-- Dockerfile
|-- docker-compose.yml
|-- README.md
|-- params.yaml                          # optional: all editable values in one file (Supplementary S5)
|-- data/
|   |-- AtTFDB_loci.txt
|   |-- biblio_marks.txt
|   |-- Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta.gz
|   `-- Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf.gz
`-- workflow/
    |-- load_libraries.R                 # R packages
    |-- run_all.sh                       # optional: runs Steps 1-3 with params.yaml (Supplementary S5)
    |-- init.R                           # init_pipeline() for R (Steps 1-2)
    |-- init.py                          # init_pipeline() for Python (Step 3)
    |-- load_libraries_python.py         # Python packages
    |-- ScRNA_Analysis_Functions.R       # helper functions (Steps 1-2)
    |-- ScRNA_Pseudotime_Functions.py    # helper functions (Step 3)
    |-- ScRNA_Pipeline_Wrappers.R        # condensed run_*() functions (version_RG, R)
    |-- ScRNA_Pseudotime_Wrappers.py     # condensed run_pseudotime() (version_RG, Python)
    |-- step0_cellranger.sh              # Part 1 - Cell Ranger (bash)
    |-- step1_singlecell.R               # Part 2 - single-cell analysis
    |-- step2_degs.R                     # Part 3 - pseudobulk DE + GO
    `-- step3_pseudotime.py              # Part 4 - pseudotime (Python)
```

The bibliography marker table (`data/biblio_marks.txt`) is part of the repository for this WT vs pifq run (see Section 1).

1.2 Container startup (Docker or Singularity)

1.2.1 Which container runtime should I use?

You need **one** container program, not both. Docker and Singularity/Apptainer run the same image and give the same results; they differ only in where they can be installed and whether they need administrator rights. Use the table to pick one, then follow **only** the matching option below and skip the other.

Where you will run the pipeline	Use	Why
Institutional cluster or shared server (Linux)	**Option A: Singularity/Apptainer**	Needs no root; Docker is usually restricted to administrators
Personal Linux computer	**Option A: Singularity/Apptainer**, or **Option B: Docker** if it is already installed	Either works; reuse what you have
Personal macOS computer	**Option B: Docker** (Docker Desktop)	Apptainer does not run natively on macOS
Personal Windows computer	**Option B: Docker** (Docker Desktop with WSL2)	Apptainer does not run natively on Windows

A personal computer is only suitable for small pilot datasets (see the requirements above). For a full run, use a cluster or server and Option A.

Check what is already installed. Run these in a terminal; a version number means the program is available, and "command not found" means it is not:

Bash

```
apptainer --version      # Option A (newer name)
singularity --version    # Option A (older name; either one is enough)
docker --version         # Option B
docker ps                # Option B only: an error here means you lack permission
```

On a cluster, Apptainer or Singularity is often installed system-wide (or available through `module avail apptainer` / `module load apptainer`). Docker may be installed but unusable if you are not in the `docker` group.

If neither is available:

- *Personal computer:* install the one chosen in the table above, following "Installing a container runtime" in the Supplementary section.
- *Cluster or shared server:* do not try to install anything. Ask the system administrators or support team, for example: "Please make Apptainer (Singularity) available so I can run the public image `docker://psblab/scrnaseq:latest`."

What is a container? A container is an isolated software environment that bundles an application with all its dependencies (libraries, packages, settings) so it runs the same way on any machine. A *container image* is the packaged snapshot you download; when you run it, it becomes a *container*. Think of the image as a recipe and the container as the dish you cook from it. This pipeline's image holds a fixed, pre-installed set of R and Python libraries — exactly the versions this pipeline was tested with. Anyone who runs the same image gets an identical environment, with no version-mismatch problems across machines, operating systems, or clusters. The container mounts the cloned repository itself as `/workspace`, so results are written back into the repository (`results/`). Since results are not pushed upstream, this is not a problem. The same public image (`psblab/scrnaseq` on Docker Hub) runs under either runtime; always pull it rather than rebuilding from the `Dockerfile`, so every user gets the exact tested package versions.

Option A: Singularity / Apptainer. This is the container app used throughout this guide. No root is required, which is what makes it the practical choice on shared servers and HPC clusters (where Docker access is usually restricted to admins). It involves two separate steps:

1. **Build the `.sif` once.** This converts the public Docker Hub image into a local Singularity image file. Only needs to be done once per machine:

Bash

```
singularity build scrnaseq.sif docker://psblab/scrnaseq:latest
```

2. **Run it with `exec`.** From the SingleCell directory cloned in Step 1.2, run:

Bash

```
singularity exec --bind $(pwd):/workspace --pwd /workspace scrnaseq.sif /bin/bash
```

This mounts the SingleCell repository as `/workspace` inside the container and sets it as the working directory. All pipeline outputs (results, figures, intermediate files) will be saved inside the repository. The container is isolated and cannot access host files without `--bind`.

If this step fails (for example with `squashfuse not found` or `Operation not permitted`), see "Troubleshooting Singularity/Apptainer" in the Supplementary section.

Apptainer imports the image environment, so the bundled Python (`/opt/venv`) and R stay on `PATH`, and it runs as the invoking user, so files under `results/` are owned by that user rather than by root.

Option B: Docker. Use this only if the table above sent you here, or if Singularity/Apptainer is not available. Requires root (or membership in the `docker` group), which is the reason it is not the default here. Mount the cloned repository and open a shell:

Bash

```
docker run -it --rm \
  -v $(pwd):/workspace \
  psblab/scrnaseq:latest /bin/bash
```

Or, from inside the cloned repository, `docker compose run r` uses the bundled `docker-compose.yml` (same image, same `/workspace` mount).

Stay inside this shell for Parts 2-4. All R and Python packages this pipeline needs are installed only inside this container image; running them outside of it fails. Start this shell inside a `tmux`/`screen` session (see above) so it survives a dropped connection.

1.3 Raw data download and Cell Ranger processing

Run this whole section inside the container, same as every other part of this pipeline: `cd` into the cloned repository and `singularity exec` into it (Section 1.2) before running any of the commands below. Write the Cell Ranger outputs into the cloned repository directory so they are visible from inside the container (which mounts that same directory as `/workspace`).

Now that the container is running, the first thing this pipeline needs is a count matrix: for every cell, how many transcripts of each gene were detected. That matrix is what the program **Cell Ranger** builds from the raw FASTQ reads, so it is the first tool run in this workflow, before anything in R or Python.

Install Cell Ranger 9.0.1 (proprietary 10x Genomics software; this workflow uses version 9.0.1). Get it from the 10x Genomics *previous versions* page:

https://www.10xgenomics.com/support/software/cell-ranger/downloads/previous-versions

1. Log in and accept the license.
2. Find **Cell Ranger 9.0.1** and copy its `curl` command.
3. That URL is signed and time-limited (it carries an `Expires=` timestamp tied to your account), so paste the current one — it cannot be hard-coded here or reused later. A plain `git clone` of the source repo does *not* give a ready-to-run binary.

Then extract it and add it to your `PATH`:

Bash

```
# Paste the signed curl command for 9.0.1 from the 10x page, e.g.:
curl -o cellranger-9.0.1.tar.gz "https://cf.10xgenomics.com/releases/cell-exp/\
cellranger-9.0.1.tar.gz?Expires=...&Key-Pair-Id=...&Signature=..."

tar -xzvf cellranger-9.0.1.tar.gz
export PATH=$PWD/cellranger-9.0.1/bin:$PATH
cellranger --version
```

The `export PATH=...` line adds Cell Ranger to the system's search path, allowing you to call `cellranger` from any directory. Verify with `cellranger --version` — it should print the version number (e.g., `cellranger-9.0.1`); `command not found` means the PATH was not set correctly or was run in a different terminal session.

The single-cell workflow starts from Cell Ranger output, specifically the `outs/filtered_feature_bc_matrix/` folder for each sample. If only raw FASTQ files are available, first download or stage the FASTQs, then run Cell Ranger for each sample before starting the R workflow.

Shortcut for a quick test. The repository already includes the Cell Ranger output of the two samples, so the FASTQ download and Cell Ranger steps can be skipped. `ScWT/` and `Scpifq/` sit at the top level of the repository and contain only the files the workflow needs: `outs/filtered_feature_bc_matrix/` (`matrix.mtx.gz`, `features.tsv.gz`, `barcodes.tsv.gz`), plus `outs/metrics_summary.csv` and `outs/web_summary.html`. They are the exact paths read by the sample manifest of Section 2, so after cloning the repository you can go directly to Part 1. The rest of this section is only needed to reproduce these matrices from the FASTQ files or to process other samples.

The WT vs pifq FASTQ files used in this guide are public data from Han et al., 2023, *Nature Plants*, deposited at the **CNCB** (China National Center for Bioinformation) Genome Sequence Archive — not NCBI — under BioProject PRJCA016521, run accession `CRA010863`. **Only two of the samples in that study are used here** (WT and pifq, listed below), not the full dataset, so results will differ from those reported in the paper:

Sample	Run accession	Files
ScWT	CRR775298	`CRR775298_f1.fastq.gz`, `CRR775298_r2.fastq.gz`
Scpifq	CRR775297	`CRR775297_f1.fastq.gz`, `CRR775297_r2.fastq.gz`

Download the reads, then rename/symlink them to the naming Cell Ranger expects (`<sample>_S1_L001_R1/R2_001.fastq.gz`)

Bash

```
mkdir -p /workspace/fastq/CRA010863

wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_f1.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_r2.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_f1.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_r2.fastq.gz

cd /workspace/fastq/CRA010863
ln -s CRR775298_f1.fastq.gz ScWT_S1_L001_R1_001.fastq.gz
ln -s CRR775298_r2.fastq.gz ScWT_S1_L001_R2_001.fastq.gz
ln -s CRR775297_f1.fastq.gz Scpifq_S1_L001_R1_001.fastq.gz
ln -s CRR775297_r2.fastq.gz Scpifq_S1_L001_R2_001.fastq.gz
```

Expected output: `ls /workspace/fastq/CRA010863` should show the four downloaded `.gz` files plus the four renamed symlinks (`ScWT_S1_L001_R1_001.fastq.gz`, etc.). Cell Ranger reads the symlink names, not the original ones, so if a rename step is skipped or misspelled, the next command below fails immediately with a "no input FASTQs found" error.

The reference genome (TAIR10.1) and Araport11 annotation ship compressed in the repository under `data/`, so no download is needed. Decompress them and build the Cell Ranger reference:

Bash

```
gunzip -k data/Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta.gz
gunzip -k data/Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf.gz

cellranger mkref \
  --genome=Ara \
  --fasta=data/Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta \
  --genes=data/Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf
```

Expected output: this creates a folder named `Ara/` (matching `--genome`) containing the indexed reference Cell Ranger needs, and ends with a line reading `>>> Reference successfully created! <<<`. This only needs to be built once; the same `Ara/` folder is reused for every sample below.

Run Cell Ranger counts for each sample:

Bash

```
cellranger count --localcores=10 --id=ScWT --fastqs=/workspace/fastq/CRA010863 \
  --sample=ScWT --transcriptome=Ara --create-bam false

cellranger count --localcores=10 --id=Scpifq --fastqs=/workspace/fastq/CRA010863 \
  --sample=Scpifq --transcriptome=Ara --create-bam false
```

What each option does:

- `--localcores=10`: number of CPU cores to use (adjust to your machine)

- `--id=ScWT`: output folder name (one per sample)
- `--fastqs=...`: directory containing the FASTQ files
- `--sample=ScWT`: sample name prefix in the FASTQ filenames
- `--transcriptome=Ara`: path to the reference built by `mkref`
- `--create-bam false`: skip BAM file generation (saves disk space; not needed for this workflow)

Expected output: each run creates a folder named after `--id` (`ScWT/`, `Scpifq/`) containing `outs/`, and ends with `Pipestance completed successfully!`. Inside `outs/`, `web_summary.html` is a self-contained QC report worth opening first (cell count, reads/cell, sequencing saturation); `filtered_feature_bc_matrix/` is the per-cell count matrix this whole pipeline is built on, and its path is what goes into the sample manifest in Section 1. This is the slowest step here, taking roughly 30-60 minutes per sample.

`--localcores=10` is used here as a portable default that keeps the example consistent across this report and the accompanying workflow scripts; it is not a hard requirement. Cell Ranger will happily use more or fewer cores, so raise or lower this value to match the number of CPU cores actually available on the machine running the pipeline (see the Recommended column above for guidance).

Check the Cell Ranger output before moving on. A poor run is much easier to diagnose here than after clustering. Open `ScWT/outs/web_summary.html` and `Scpifq/outs/web_summary.html` in a browser. To do this: if working on a local machine, double-click the file or drag it into a browser window; if working on a remote server, copy the file to your local machine with `scp` or mount the remote directory, then open it locally. The same numbers are in `outs/metrics_summary.csv` (plain text, viewable with `cat` or any text editor). Look at the following:

Metric	What it tells you	What to look for
Estimated number of cells	Barcodes called as cells	Close to the number of cells you intended to capture; very different from the expected value suggests a loading or calling problem
Mean reads per cell / median genes per cell	Sequencing depth and per-cell complexity	Both should be of the same order in the two samples; a large imbalance can introduce a technical difference between WT and pifq
Sequencing saturation	How much deeper sequencing would still add new transcripts	Low values mean more sequencing would recover more genes; very high values mean little is gained
Valid barcodes and reads in cells	Library quality and how much signal is in real cells versus ambient background	Low values indicate a poor library or a high background
Reads mapped confidently to the transcriptome	Reference suitability	A low fraction can mean a wrong or incomplete reference or annotation

The **barcode rank plot** (the "knee plot", under the *Cells* panel) ranks barcodes by total UMI counts. A good run shows a sharp drop (the "knee") separating a high-count group of cells from a long tail of empty droplets. A gradual slope with no clear knee, or a knee at a very different position in the two samples, means that cell calling was uncertain and the cell counts of the two samples may not be comparable. These values are not strict pass/fail thresholds, and they depend on the tissue and the protocol; use them to compare the two samples with each other and to spot samples that look clearly different from the other.

Cell Ranger's cell calling is only a first pass: the QC filtering in Part 2 (Section 2) removes low-quality cells and doublets from this set, so the cell number reported here is expected to be larger than the one used downstream.

2 Part 2 - Single-cell analysis in R

From this point on, the pipeline runs in R inside the container. Start R:

Bash

```
R
```

Note: the processed matrices are already in the repository. The Cell Ranger output of both samples is included in the repository, so this part can be started directly from here, without running Part 1. The files read by the sample manifest are `ScWT/outs/filtered_feature_bc_matrix/` and `Scpifq/outs/filtered_feature_bc_matrix/`.

2.1 Initialization

A single call to `init_pipeline()` loads the R packages and helper functions, sets the seed, and creates the output folders. `data_dir` is the project folder (the cloned repository, mounted as `/workspace`); `seed` is the random seed. The second `source()` loads the condensed functions used in the rest of this guide.

R

```r
source("/workspace/workflow/init.R")
init_pipeline(data_dir = "/workspace/.", seed = 1807)
source("/workspace/workflow/ScRNA_Pipeline_Wrappers.R")
```

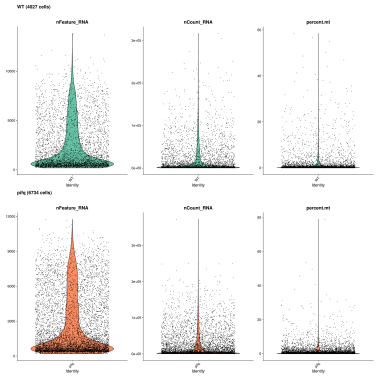
2.2 Sections 1-4 - Preprocessing: QC, filtering, merging and Harmony

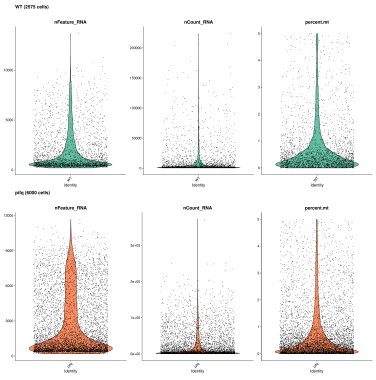
`run_preprocessing()` runs, in order: data loading and pre-filter QC (Section 1), cell filtering and doublet detection (Section 2), merging, normalization and PCA (Section 3), and Harmony batch correction (Section 4). It saves the QC violin plots and the UMAPs before and after Harmony to `results/01_qc/`, and the intermediate objects (`seurat_list_postfilter.rds`, `ath_sc_preharmony.rds`, `ath_sc_postharmony.rds`) to `results/objects/`.

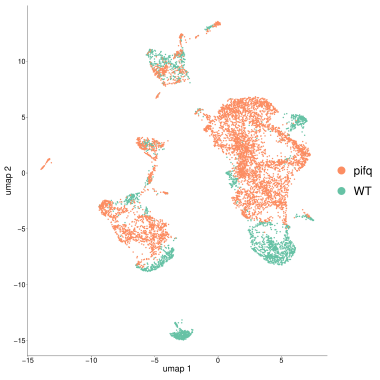
- `samples`: one entry per sample, with the path of its `filtered_feature_bc_matrix/` folder (relative to `data_dir`), a unique `label` and the experimental `condition`.
- `colors`: one color per sample label.
- `mt_pattern`: regular expression for mitochondrial genes (`"^ATMG"` for Arabidopsis, `"^MT-"` for human, `"^mt-"` for mouse).
- `min_features`, `max_mt`, `run_doubletfinder`: filtering thresholds.
- `nfeatures`, `npcs`, `dims`: variable features, principal components, and dimensions used by Harmony/UMAP.

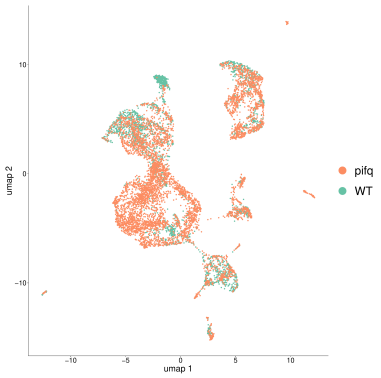
R

```r
samples <- list(
  list(file = "ScWT/outs/filtered_feature_bc_matrix",   label = "WT",   condition = "WT"),
  list(file = "Scpifq/outs/filtered_feature_bc_matrix", label = "pifq", condition = "pifq")
)
colors <- c("WT" = "#66c2a5", "pifq" = "#fc8d62")
pre <- run_preprocessing(
  samples, colors,
  mt_pattern        = "^ATMG",
  min_features      = 200,
  max_mt            = 5,
  run_doubletfinder = TRUE,
  nfeatures         = 2000,
  npcs              = 30,
  dims              = 1:30
)
ath_sc <- pre$ath_sc
```







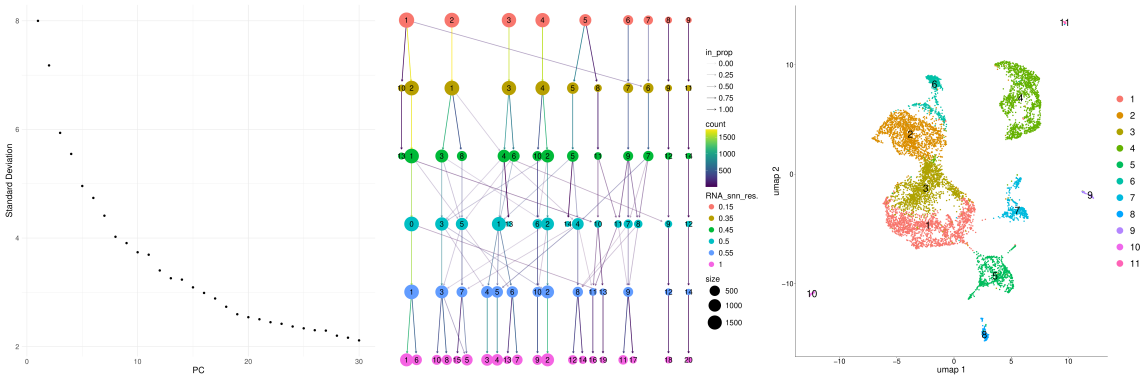

umap 2
umap 1
pifq
WT

2.3 Sections 5-6 - Clustering resolution and final clustering

`run_clustering()` saves the elbow plot and the clustree of the candidate resolutions (Section 5) to `results/02_clustering/`, and then clusters the cells at `cluster_resolution` (Section 6), saving the UMAP of the clusters. Review the elbow plot and the clustree, and rerun with a different `cluster_resolution` if needed.

R

```r
ath_sc <- run_clustering(
  ath_sc,
  resolutions_test   = c(0.15, 0.35, 0.45, 0.55, 1.0),
  cluster_resolution = 0.35,
  dims               = 1:30
)
```

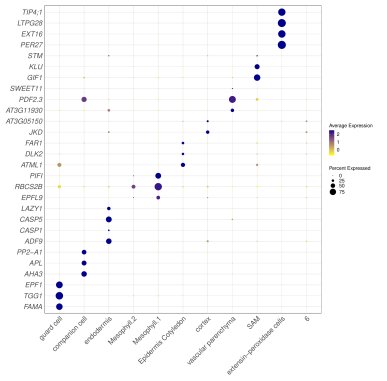


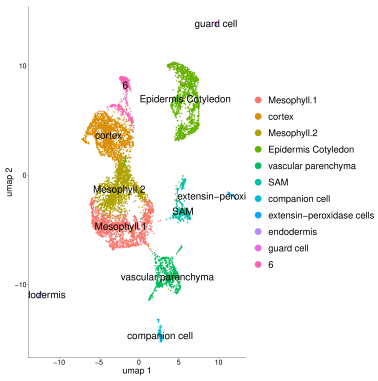
2.4 Sections 7-8 - Annotation and gene expression

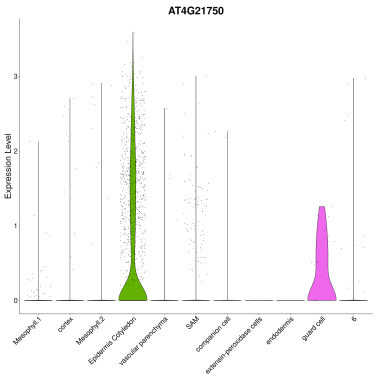
`run_annotation()` finds the cluster markers, annotates the cell types from the bibliography marker table (Section 7), and plots the expression of `gene` (Section 8, optional: use `gene = NULL` to skip it). Results go to `results/03_annotation/` and `results/04_expression/`.

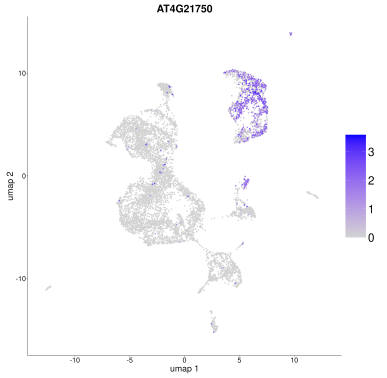
R

```
ath_sc <- run_annotation(
  ath_sc,
  marker_file = "data/biblio_marks.txt",
  gene        = "AT4G21750"
)
```









2.5 Section 9 - Export the annotated object

The annotated object is saved as an `.rds` checkpoint and exported to AnnData (`.h5ad`) for the Python trajectory analysis. Both files are written to `results/objects/`. Restore the checkpoint in a later session with `ath_sc <- readRDS(file.path(dir_objects, "ath_sc_annotated.rds"))`.

R

```
saveRDS(ath_sc, file.path(dir_objects, "ath_sc_annotated.rds"))
export_to_scanpy(
  ath_sc,
  file.path(dir_objects, "ath_sc_annotated.h5ad")
)
message("\\nOK SECTION 9 COMPLETE: annotated object saved (.rds + .h5ad)")
```

3 Part 3 - Pseudobulk differential expression and GO

This part continues from the annotated object. It can also be started from a new R session, after the initialization block above, by loading the object:

R

```
ath_sc <- readRDS(file.path(dir_objects, "ath_sc_annotated.rds"))
```

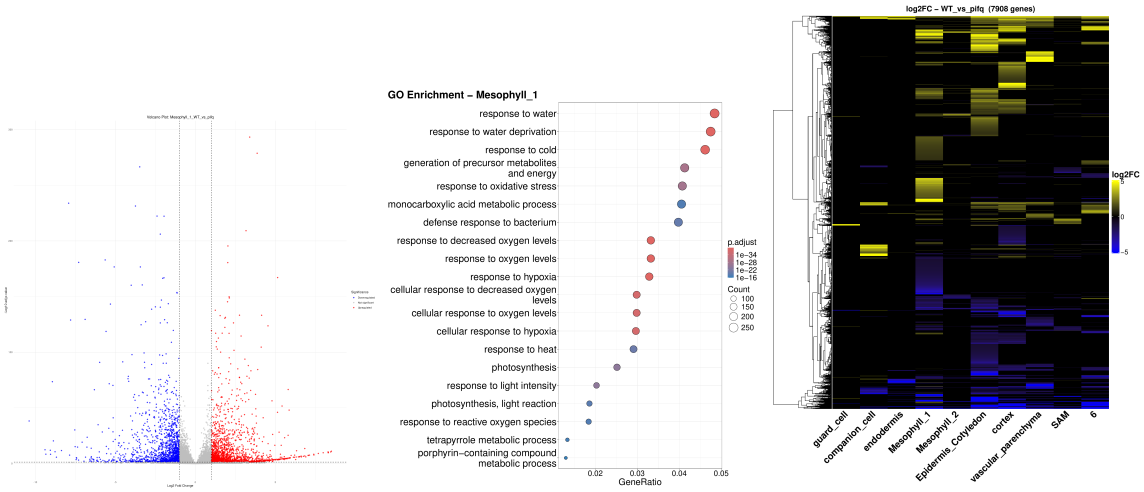
3.1 Sections 10-16 - Pseudobulk, DESeq2, volcano, tables, GO and heatmap

`run_pseudobulk_de()` creates the cell-type subsets (Section 10), assigns pseudo-replicates (Section 11), aggregates counts and runs DESeq2 (Section 12), and then, for each comparison, draws the volcano plots (Section 13), builds the differential gene tables (Section 14), runs the GO enrichment (Section 15) and draws the log2FC heatmap (Section 16). Results go to `results/06_de_results/` and `results/07_go/`.

- `comparisons`: list of contrasts; each has the two conditions and a `tag`.
- `n_pseudoreps`: pseudo-replicates per condition and cell type.
- `padj_cut`, `lfc_cut`: significance and log2 fold-change thresholds.
- `go_space`, `go_orgdb`, `go_keytype`: GO ontology, annotation package and gene ID type.

R

```
de <- run_pseudobulk_de(
  ath_sc,
  marker_file    = "data/biblio_marks.txt",
  comparisons    = list(list(conds = c("WT", "pifq"), tag = "WT_vs_pifq")),
  annot_col      = "celltype",
  n_pseudoreps   = 3,
  padj_cut       = 0.05,
  lfc_cut        = 1,
  go_space       = "BP",
  go_orgdb       = org.At.tair.db,
  go_keytype     = "TAIR",
  heatmap_limits = c(-5, 5)
)
```



3.2 Section 17 - Gene-gene co-expression network (hdWGCNA) and TF network

`run_tf_network()` builds the hdWGCNA co-expression network for each cell type and the TF network, colored by the direction of differential expression of the chosen `contrast_tag`. Results go to `results/08_networks/`.

R

```r
run_tf_network(
  ath_sc,
  contrast_tag    = "WT_vs_pifq",
  annot_col       = "celltype",
  tf_list         = "data/AtTFDB_loci.txt",
  n_metacells     = 50,
  min_module_size = 20,
  deep_split      = 2,
  soft_power      = NULL,
  tom_threshold   = 0.2,
  n_hub_label     = 15
)
```

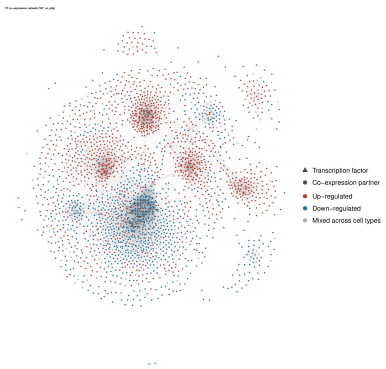


Figure 1: TF co-expression network, colored by DE direction.

4 Part 4 - Pseudotime trajectory analysis (Python)

Part 4 runs in Python (scanpy, Palantir, scFates), starting from the h5ad object exported at the end of Part 2. Exit R and start Python inside the same container:

Bash

```
q()           # leave R (do not save the workspace)
python3
```

4.1 Section 18 - Setup

A single call to `init_pipeline()` loads the Python packages and helper scripts and creates the pseudotime output folder. The second line loads the condensed `run_pseudotime()` function.

Python

```
exec(open("/workspace/workflow/init.py").read(), globals())
init_pipeline(data_dir="/workspace/.")
exec(open("/workspace/workflow/ScRNA_Pseudotime_Wrappers.py").read(), globals())
```

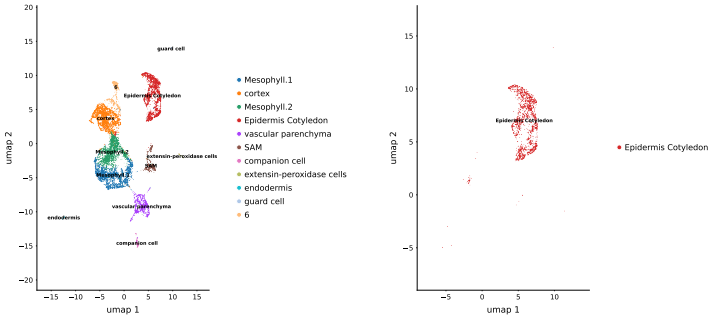
4.2 Sections 19-23 - Trajectory inference and gene trends

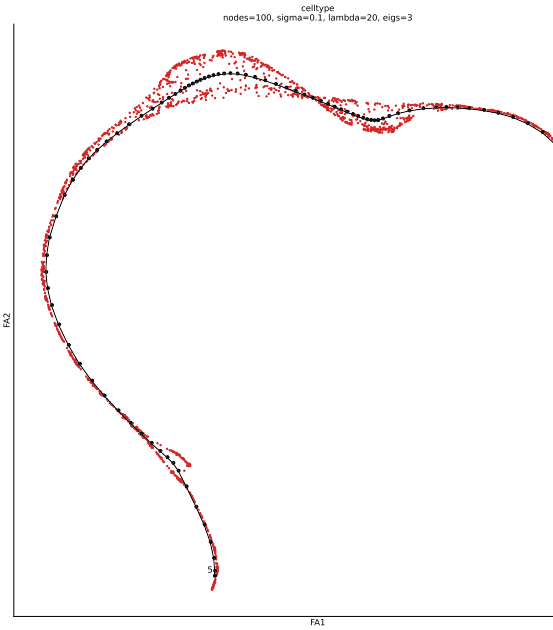
`run_pseudotime()` loads the annotated object (Section 19), selects the cell types (Section 20), infers the trajectory (Section 21), plots genes on the trajectory (Section 22, optional) and computes and plots the gene trends (Section 23). Results go to `results/09_pseudotime/`.

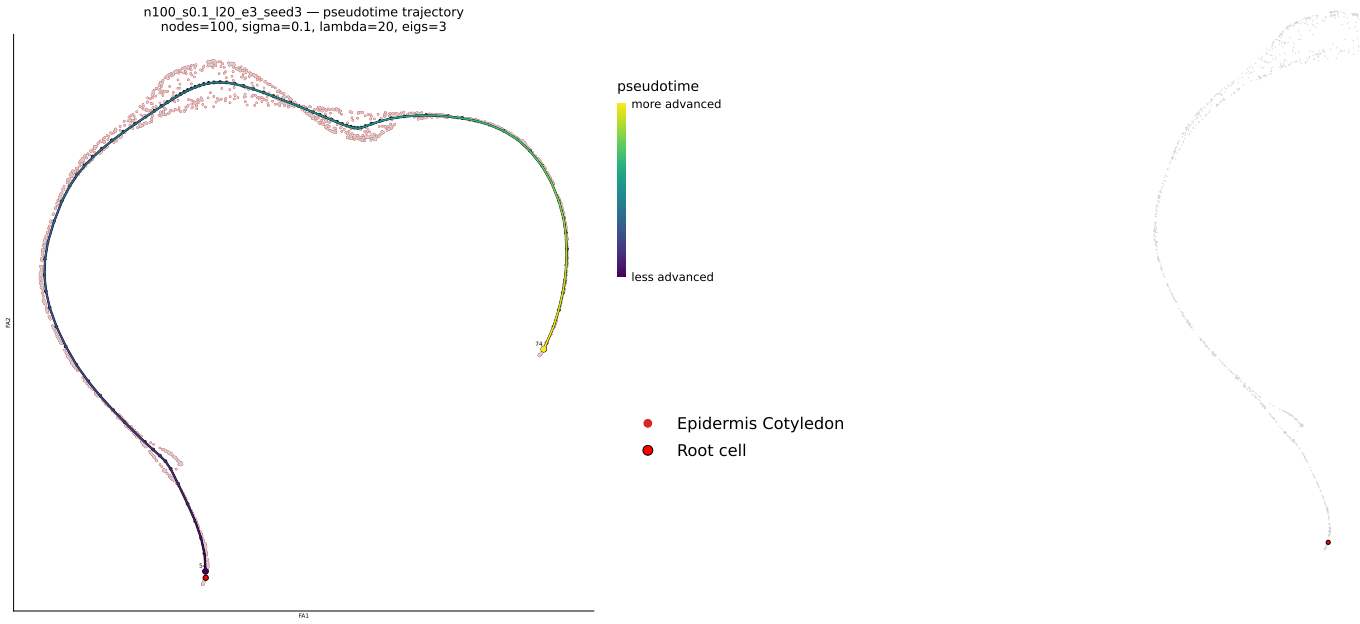
- `input_h5ad`, `annotation_col`: the exported object and the annotation column (`celltype`, `celltype_grouped` or `celltype_curated`, depending on the Supplementary Sections used).
- `clusters`: cell types used for the trajectory; `root_cluster`: root cell type. With a single cell type, the root is set automatically at the tip of the tree.
- `runs`: one or more `trajectory_run()` parameter sets (nodes, sigma, lambda, eigs, seed).
- `plot_genes`, `plot_titles`, `plot_filename`: genes drawn on the trajectory (use `plot_genes=None` to skip).
- `spline_df`, `top_n`, `highlight_genes`, `ordering`: gene-trend fit and plot. The fit is computed once and saved, so changing the plot arguments only requires rerunning this call with the saved fit.

Python

```
adata_traj, selected_trajectory_dir, adata_fitted = run_pseudotime(
    input_h5ad      = "/workspace/results/objects/ath_sc_annotated.h5ad",
    annotation_col  = "celltype",
    clusters        = ["Epidermis Cotyledon"],
    root_cluster    = "Epidermis Cotyledon",
    runs            = [trajectory_run(nodes=100, sigma=0.1, lambda_value=20, eigs=3, seed=3)],
    n_jobs          = 4,
    plot_genes      = ["AT4G21750"],
    plot_titles     = ["ATML1 (AT4G21750)"],
    plot_filename   = "trajectory_genes_ATML1.pdf",
    spline_df       = 3,
    top_n           = 10,
    highlight_genes = ["AT4G21750"],
    ordering        = "max",
)
```







4.3 Section 24 - Final export

The final pseudotime object, with the trajectory, pseudotime values and layout coordinates, is saved in `results/objects/` as `ath_sc_pseudotime_final.h5ad`.

Python

```
final_h5ad = os.path.join(base_dir, "objects", "ath_sc_pseudotime_final.h5ad")
os.makedirs(os.path.dirname(final_h5ad), exist_ok=True)
adata_traj.write_h5ad(final_h5ad)
print(f"SECTION 24 COMPLETE: final pseudotime object saved -> {final_h5ad}")
```

This is the last step of the pipeline. Leave the Python REPL by exiting it explicitly, mirroring `q()` on the R side:

Bash

```
exit()        # leave Python
```

Output layout {-}

All four parts write into the same `results/` folder inside the cloned repository (mounted as `/workspace/results` in the container), one subfolder per step. After running the full pipeline once, it looks like this:

```{=latex}
\scriptsize
```

```
results/
|-- 01_qc/
|   |-- qc_prefilter.pdf
|   |-- qc_postfilter.pdf
|   |-- umap_preharmony.pdf
|   `-- umap_postharmony.pdf
|-- 02_clustering/
|   |-- elbow_plot.pdf
|   |-- clustree.pdf
|   `-- umap_seuratclusters.pdf
|-- 03_annotation/
|   |-- FindAllMarkers.tsv
|   |-- dotplot_marker_table_annotation_biblio.pdf
|   |-- umap_annotation_biblio.pdf
|   `-- clustree_annotated.pdf
|-- 04_expression/
|   |-- feature_gene_<gene>.pdf
|   `-- vln_gene_<gene>.pdf
|-- 05_curation/
|   |-- umap_grouped.pdf
|   |-- subcluster_markers_<cluster>.pdf
|   `-- umap_curated.pdf
|-- objects/                              # checkpoints (.rds / .h5ad)
|   |-- seurat_list_postfilter.rds
|   |-- ath_sc_preharmony.rds
|   |-- ath_sc_postharmony.rds
|   |-- ath_sc_annotated.rds
|   |-- ath_sc_annotated.h5ad
|   |-- pseudobulk_replicates/
|   `-- ath_sc_pseudotime_final.h5ad
|-- 06_de_results/<comparison_tag>/       # e.g. WT_vs_pifq/
|   |-- DESeq2_<cell_type>_<tag>.csv
|   |-- <cell_type>_<tag>_filtered.csv
|   |-- diff_table_<tag>_fc1_padj_005.tsv
|   |-- log2FC_table_fc1_padj_005.tsv
|   |-- heatmap_<tag>.pdf
|   `-- volcano/
|       |-- DESeq2_<cell_type>_<tag>.png
|       `-- VolcanoPlots_<tag>.pdf
|-- 07_go/<comparison_tag>/
|   |-- GO_<cell_type>_<tag>_BP.tsv
|   `-- GO_<cell_type>_<tag>_BP_bubble.pdf
|-- 08_networks/
|   |-- TF_TARGET_interaccion.tsv
```

```
|   |-- edges_unified.tsv
|   |-- nodes_unified.tsv
|   |-- modules_unified.tsv
|   |-- hubgenes_unified.tsv
|   |-- hdwgcna_summary.tsv
|   |-- unified_TOM.rda
|   |-- hdwgcna_unified.rds
|   `-- network_tf_DE_direction.pdf
`-- 09_pseudotime/
    |-- umap_overview.pdf
    |-- umap_selection.pdf
    `-- trajectory/<run_name>/            # e.g. n100_s0.1_l20_e3_seed3/
        |-- parameters.tsv
        |-- <run_name>_annotation.pdf
        |-- <run_name>_pseudotime_trajectory.pdf
        |-- <run_name>_root_cell.pdf
        |-- <run_name>_pseudotime_by_cell.tsv
        |-- <run_name>_trajectory.h5ad
        |-- gene_plots/trajectory_gene_<gene>.pdf
        `-- gene_trends/
            |-- step29_gene_trends_top10_FINAL_24x18.pdf
            `-- step29_gene_trends_highlight_list_FINAL_24x18.pdf
```

`01_qc-05_curation` and `objects/` come from Part 2 (`step1_singlecell.R`); `06_de_results/07_go` (one subfolder per `comparisons` tag, Section 12) and `08_networks` from Part 3 (`step2_degs.R`); `09_pseudotime` from Part 4 (`step3_pseudotime.py`), one `trajectory/<run_name>/` per entry in `TRAJECTORY_RUNS` (Section 21). `<gene>`, `<cluster>`, `<cell_type>`, and `<tag>` depend on the values used when running each section, not on fixed names.

Supplementary methods

The three steps below are optional. They start from the annotation of Section 7 and write their results to separate columns (`celltype_grouped`, `celltype_curated`) or figures, so the main workflow runs without them. If you use one of them, set the annotation column of Supplementary Section S2 accordingly. The code is also available in `workflow/step1_supplementary.R`.

Supplementary Section S1 - Annotated clustree

This section adds the cell-type labels from Section 7 onto the clustering tree from Section 5. Instead of showing only numeric clusters, each node is labeled with the most common cell type found in that cluster.

This makes it easier to see how stable the annotated cell types are across different clustering resolutions. The output is saved as `results/03_annotation/clustree_annotated.pdf`.

R

```
output_dir <- dir_03
plot_annotated_clustree(
  ath_sc, clu, annot_col = "celltype", output_dir = output_dir
)
message("\nOK SUPPLEMENTARY S1 COMPLETE: annotated clustree saved")
```

Representative output from Supplementary Section S1:

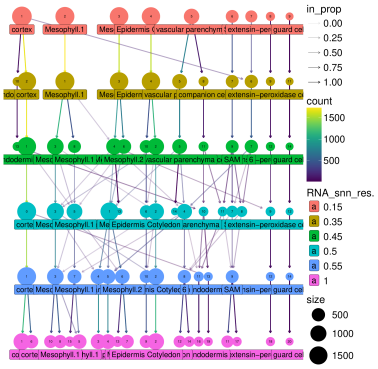


Figure 2: Cluster tree labeled by the dominant bibliography-based annotation in each node.

Supplementary Section S2 - Cell-type grouping

This optional section simplifies annotation labels by merging selected cell types or subclusters into broader groups. It uses the `grouping` object as a small renaming table: the names on the left are the current labels, and the values on the right are the new labels.

For example, in this report the two mesophyll subclusters, `Mesophyll.1` and `.2`, are renamed as `Mesophyll`. Any cell type that is not listed in `grouping` keeps its original label.

Use this section when the detailed annotation is useful for inspection but a simpler grouping is better for figures or downstream summaries. The grouped labels are stored in `celltype_grouped`, and the grouped UMAP is saved as `results/05_curation/umap_grouped.pdf`.

R

```r
# Merge the two mesophyll subclusters into a single label
grouping <- c(
  "Mesophyll.1" = "Mesophyll",
  "Mesophyll.2" = "Mesophyll"
)

output_dir <- dir_05

if (length(grouping) > 0) {
  current <- as.character(ath_sc$celltype)
  ath_sc$celltype_grouped <- ifelse(current %in% names(grouping), grouping[current], current)
} else {
  ath_sc$celltype_grouped <- ath_sc$celltype
}

save_pdf(
  plot_umap(ath_sc, "celltype_grouped"),
  "umap_grouped.pdf"
)

message("\nOK SUPPLEMENTARY S2 COMPLETE: cell-type grouping complete")
```

Representative output from Supplementary Section S2:

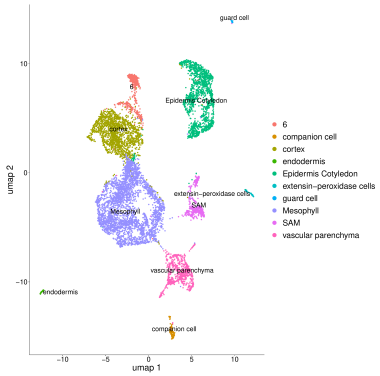


Figure 3: UMAP colored by grouped cell-type labels.

Supplementary Section S3 - Subcluster inspection and manual curation

This optional section is for manual curation when one annotated cluster needs a closer look. First, `inspect_subcluster_markers()` takes one cluster, splits it into smaller subclusters, and saves a figure with the subcluster UMAP and marker gene patterns. This helps decide whether the original label should be kept or refined.

If refinement is needed, `reassign` is edited as a small mapping table. The outer name is the cluster being curated, the names inside are subcluster IDs, and the values are the new labels. The special label `others` is used for any subcluster not listed explicitly.

In this example the inspected cluster is `Mesophyll.1`, one of the two mesophyll subclusters that Supplementary Section S2 merges into `Mesophyll`. The inspection confirms the original label, so `reassign` maps every

Mesophyll subcluster back to `"Mesophyll"` and no cell is relabeled; this is the pattern to use when the inspection shows that the original annotation was already correct.

The names of `reassign` are looked up in the column given by `annot_col`. Because `Mesophyll` is a label of `celltype_grouped` (created in Supplementary Section S2) and not of `celltype`, which only has `Mesophyll.1` and `Mesophyll.2`, the call uses `annot_col = "celltype_grouped"`; S2 must be run first, otherwise the function stops with a message that says so.

Then `curate_clusters()` applies those manual labels and stores the result in `celltype_curated`. The final curated UMAP is saved as `results/05_curation/umap_curated.pdf`. This block is not executed automatically because the label choices depend on inspecting the figures.

R

```r
output_dir <- dir_05
Idents(ath_sc) <- "celltype"

# 1. Inspect a cluster (saves the combined subcluster + marker figure)
inspect_subcluster_markers(
  ath_sc, cluster_id = "Mesophyll.1",
  marker_table = marker_table, output_dir = output_dir
)

# 2. Map each cluster's subclusters to labels ("others" = any ID not listed),
#    then curate them in one call. Here Mesophyll's subclusters all map back
#    to "Mesophyll" (no relabeling), which is the pattern to use when the
#    subcluster inspection confirms the original label was already correct.
#    The keys of `reassign` are labels of celltype_grouped (created in
#    Supplementary Section S2), so annot_col and source_col point to that column:
reassign <- list(
  "Mesophyll" = c("others" = "Mesophyll")
)
ath_sc <- curate_clusters(
  ath_sc, reassign,
  marker_table = marker_table, output_dir = output_dir,
  annot_col = "celltype_grouped", source_col = "celltype_grouped"
)

# 3. Save the curated annotation UMAP
save_pdf(
  plot_umap(ath_sc, "celltype_curated"),
  "umap_curated.pdf"
)

message("\nOK SUPPLEMENTARY S3 COMPLETE: curation complete")
```

Representative output from Supplementary Section S3 — subcluster inspection of the Mesophyll population (UMAP of its subclusters plus marker-gene panels used to decide whether to keep or split the label):

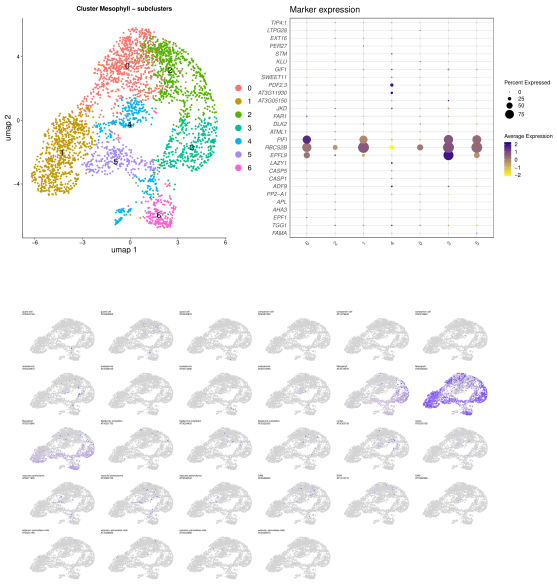


Figure 4: Mesophyll subcluster inspection: subcluster UMAP, marker dotplot, and per-gene marker panels.

In this example the Mesophyll subclusters are mapped back to `Mesophyll` *(no relabeling), which shows the inspect-then-curate mechanism without changing any label. The main workflow does not need this step: it continues with the labels from Section 7.*

Supplementary Section S4 - Manual root override

If the automatic root selection (Section 21) does not match the expected biological starting point, the root can be set manually by specifying a tree node number. This requires two runs:

1. **First run with `auto`:** Run the trajectory with default settings. The pipeline generates an FA (ForceAtlas2) plot showing the tree structure with numbered nodes at each tip. Inspect this plot to identify which tip corresponds to the biological root (e.g., the progenitor population).
2. **Second run with `tip`:** Re-run with `root_method = "tip"` and `root_tip` set to the node number identified in step 1.

Python

```python
# Example: re-run with manual root at node 42
adata_traj, selected_trajectory_dir, trajectory_runs = run_trajectory_runs(
    adata           = adata,
    clusters        = TRAJECTORY_CLUSTERS,
    root_cluster    = ROOT_CLUSTER,
    annotation_col  = ANNOTATION_COL,
    root_method     = "tip",
    root_tip        = 42,  # node number from FA plot
)
```

This approach is useful when the Gulati score (number of detected genes) does not correctly identify the progenitor population, for example in datasets where differentiated cells have higher transcriptional diversity than progenitors.

Supplementary Section S5 - Running the workflow from a parameter file

This section is optional. The workflow is run exactly as described in the previous sections: every value that you may want to change is written in the code blocks of this document and of `workflow/step*.R|py`.

The repository also contains `params.yaml`, a single file that gathers all those values, grouped by step: project folder, seed and results folder (`project`), names of the objects passed between steps (`outputs`), samples, colors and QC thresholds, resolutions, annotation file, genes, pseudobulk comparisons, GO and network settings, the optional supplementary sections S2-S3, and the trajectory settings of the Python step (`pseudotime`). It is meant for running the workflow with other values without editing the scripts, for example to repeat it on other samples.

The scripts of `workflow/` do not read `params.yaml`. The tool `workflow/make_param_steps.py` makes, from them, copies in which every editable value is read from `params.yaml` and, if the file does not give it, keeps the value of the script. The copies are generated every time, so they always match the scripts. `workflow/run_all.sh` generates them and runs the analysis steps (R, R, Python):

Bash

```bash
# all analysis steps (1, 2 and 3), one after the other, with params.yaml
bash workflow/run_all.sh

# only some steps
bash workflow/run_all.sh "2 3"

# a single step: generate the copies, then run the one you want
python3 workflow/make_param_steps.py results/params_steps
PARAMS_FILE=/workspace/params.yaml Rscript results/params_steps/step1_singlecell.R
```

Each step writes its log to `results/logs/stepN.log`, and the run stops at the first step that fails. Step 0 (Cell Ranger) is not part of the default run because it needs a signed 10x download URL, and it does not read `params.yaml`; `step1_supplementary.R` is not run either, because it continues the R session of Step 1 (its generated copy is in the same folder, for manual use). A run with a parameter file does not stop to look at figures, so the resolution of Section 6 and the annotation file of Section 7 are the ones written in the file: for new data, look at the elbow plot and the clustree (Section 5) first and set `clustering: cluster_resolution` accordingly. The Python step needs PyYAML to read the file (`pip install pyyaml`).

Supplementary Table — Software dependencies and versions

Package versions are those bundled in the `psblab/scrnaseq:latest` Docker image (the public image used for all R and Python steps). R **base-priority** packages are omitted. Cell Ranger is 10x Genomics proprietary software run *outside* the container (Part 1); its version is fixed by the workflow.

Interpreters and command-line tools

Tool	Version
Cell Ranger	9.0.1
R	4.5.3
Python	3.12.3

Installing tmux or screen

tmux: see the official installation guide at https://github.com/tmux/tmux/wiki/Installing maintained by the tmux developers. This is the authoritative source for platform-specific instructions (Linux package managers, macOS Homebrew/MacPorts, BSD ports, building from source, and Windows via WSL2). The wiki is kept up-to-date with each release and covers edge cases not listed here.

screen: `sudo apt install screen` (Debian/Ubuntu), `sudo yum install screen` (RHEL/CentOS/Fedora), or ships with macOS by default.

Windows users: this whole pipeline (Cell Ranger, Docker/Singularity, R, Python) is Linux-based. The practical options on Windows are WSL2 (https://learn.microsoft.com/windows/wsl/install) running Ubuntu locally, or an SSH client (Windows Terminal, PuTTY, or WSL2's own `ssh`) connecting out to a Linux server or HPC cluster where the rest of this guide applies directly. Once inside WSL2 or an SSH session on Linux, use the commands above.

Installing a container runtime (Docker or Singularity/Apptainer)

Pick one; both run the same `psblab/scrnaseq:latest` image (see "Container startup" in Part 1).

Docker

OS	Install
Linux	https://docs.docker.com/engine/install/ (distro-specific packages)
macOS	Docker Desktop: https://docs.docker.com/desktop/install/mac-install/
Windows	Docker Desktop with the WSL2 backend: https://docs.docker.com/desktop/install/windows-install/

Singularity / Apptainer (no root required; common on HPC clusters where Docker is not permitted)

OS	Install
Linux	https://apptainer.org/docs/admin/main/installation.html
macOS / Windows	not natively supported — use a Linux VM, WSL2, or the remote server/cluster itself

On a shared HPC cluster, Singularity/Apptainer is usually already installed system-wide — check with `singularity --version` or `apptainer --version` before installing anything.

Troubleshooting Singularity/Apptainer

Symptom. `singularity build` finishes, but the startup command (`singularity exec ... scrnaseq.sif /bin/bash`) fails with `squashfuse not found` followed by `Operation not permitted` during extraction.

Cause. A `.sif` file is a compressed (squashfs) filesystem that must be mounted to run. Where Apptainer is installed with its setuid helper (common on shared clusters), this works with no further setup. In a rootless installation without that helper, Apptainer mounts the image through FUSE using `squashfuse`; if it is missing, Apptainer falls back to extracting the image, which fails when user namespaces are disabled or restricted, or when the temporary directory is unusable.

Prerequisite check. Run these three commands:

Bash

```
apptainer --version                          # must print a version number
which squashfuse fusermount                  # must print a path for both programs
cat /proc/sys/user/max_user_namespaces       # must print a number greater than 0
```

The first command must always work. If Apptainer was installed with its setuid helper (common on shared clusters), nothing else is needed. Otherwise the second command must print a path for both programs, and the third must print a number greater than 0. If either is missing, the error above is expected.

Fixes, in order of effort.

1. Install `squashfuse` and `fuse` (on a cluster, ask the administrators).
2. Point Apptainer to a local disk with free space that is not mounted `noexec`, then retry:

Bash

```
export APPTAINER_TMPDIR=/path/to/local/disk/tmp
export APPTAINER_CACHEDIR=/path/to/local/disk/cache
mkdir -p "$APPTAINER_TMPDIR" "$APPTAINER_CACHEDIR"
```

3. Avoid mounting by building a directory instead of a `.sif`, and run that:

Bash

```
apptainer build --sandbox scrnaseq/ docker://psblab/scrnaseq:latest
apptainer exec --bind $(pwd):/workspace --pwd /workspace scrnaseq/ /bin/bash
```

4. If none of these work, contact your system administrators, or use Docker (Option B) on a machine where it is permitted.

On Ubuntu 24.04 and similar distributions, AppArmor can restrict unprivileged user namespaces by default; this requires an administrator to allow Apptainer.

R packages (421)

Note on package versions: The tables below list the package versions installed in the `psblab/scrnaseq:latest` container image at the time of writing. These versions are locked inside the container, so anyone who pulls the image gets exactly these versions — there is nothing to install manually. **The authoritative source is the container itself**, not this list; if the image is updated in the future, the container's installed versions take precedence. To inspect the current versions inside a running container, use `R -e "installed.packages()[,3]"` for R or `pip list` for Python.

Package	Version	Package	Version
abind	1.4-8	locfit	1.5-9.12
anndataR	1.0.2	lubridate	1.9.5
AnnotationDbi	1.72.0	magrittr	2.0.5
ape	5.8-1	maps	3.4.3
aplot	0.2.9	MASS	7.3-65
askpass	1.2.1	Matrix	1.7-5
assertthat	0.2.1	MatrixGenerics	1.22.0
assorthead	1.4.0	MatrixModels	0.5-4
backports	1.5.1	matrixStats	1.5.0
base64enc	0.1-6	memoise	2.0.1
basilisk	1.22.0	mgcv	1.9-4
batchelor	1.26.0	microbenchmark	1.5.0
beachmat	2.26.0	mime	0.13
beeswarm	0.4.0	miniUI	0.1.2
BH	1.90.0-1	minqa	1.2.8
biglm	0.9-3	modelr	0.1.11
Biobase	2.70.0	monocle3	1.4.27
BiocGenerics	0.56.0	nlme	3.1-169
BiocManager	1.30.27	nloptr	2.2.1
BiocNeighbors	2.4.0	nnet	7.3-20
BiocParallel	1.44.0	numDeriv	2016.8-1.1
BiocSingular	1.26.1	openssl	2.4.1
BiocVersion	3.22.0	org.At.tair.db	3.22.0
Biostrings	2.78.0	otel	0.2.0
bit	4.6.0	pak	0.9.5
bit64	4.8.2	parallelly	1.47.0
bitops	1.0-9	patchwork	1.3.2
blob	1.3.0	pbapply	1.7-4
boot	1.3-32	pbkrtest	0.5.5
BPCells	0.3.1	pbmcapply	1.5.1
brew	1.0-10	pheatmap	1.0.13
brio	1.1.5	pillar	1.11.1
broom	1.0.13	pkgbuild	1.4.8
bslib	0.11.0	pkgconfig	2.0.3
cachem	1.1.0	pkgdown	2.2.0
Cairo	1.7-0	pkgload	1.5.2
callr	3.7.6	plotly	4.12.0
car	3.1-5	plyr	1.8.9
carData	3.0-6	png	0.1-9
caTools	1.18.3	polyclip	1.10-7
cellranger	1.1.0	polynom	1.4-1
checkmate	2.3.4	praise	1.0.0
circlize	0.4.18	preprocessCore	1.72.0
class	7.3-23	prettyunits	1.2.0
classInt	0.4-11	processx	3.9.0
cli	3.6.6	profvis	0.4.0
clipr	0.8.1	progress	1.2.3
clue	0.3-68	progressr	0.19.0
cluster	2.1.8.2	promises	1.5.0
clusterProfiler	4.18.4	proxy	0.4-29
clustree	0.5.1	ps	1.9.3
codetools	0.2-20	pscl	1.5.9
colorspace	2.1-2	purrr	1.2.2
commonmark	2.0.0	quadprog	1.5-8
ComplexHeatmap	2.26.1	quantreg	6.1
conflicted	1.2.0	qvalue	2.42.0
corrplot	0.95	R.methodsS3	1.8.2
cowplot	1.2.0	R.oo	1.27.1
cpp11	0.5.5	R.utils	2.13.0
crayon	1.5.3	R6	2.6.1
credentials	2.0.3	ragg	1.5.2
crosstalk	1.2.2	RANN	2.6.2
curl	7.1.0	rappdirs	0.3.4
data.table	1.18.4	rbibutils	2.4.1
DBI	1.3.0	rcmdcheck	1.4.0
dbplyr	2.5.2	RColorBrewer	1.1-3

Package	Version	Package	Version
DelayedArray	0.36.1	Rcpp	1.1.1-1.1
DelayedMatrixStats	1.32.0	RcppAnnoy	0.0.23
deldir	2.0-4	RcppArmadillo	15.2.7-1
Deriv	4.2.0	RcppEigen	0.3.4.0.2
desc	1.4.3	RcppHNSW	0.7.0
DESeq2	1.50.2	RcppML	0.3.7.1
devtools	2.5.2	RcppProgress	0.4.2
diffobj	0.3.6	RcppRoll	0.3.2
digest	0.6.39	RcppTOML	0.2.3
dir.expiry	1.18.0	Rdpack	2.6.6
distributional	0.7.0	readr	2.2.0
doBy	4.7.1	readxl	1.5.0
docopt	0.7.2	reformulas	0.4.4
doParallel	1.0.17	rematch	2.0.0
DOSE	4.4.0	rematch2	2.1.2
dotCall64	1.2	remotes	2.5.0
DoubletFinder	2.0.6	reprex	2.1.1
downlit	0.4.5	reshape2	1.4.5
dplyr	1.2.1	ResidualMatrix	1.20.0
dqrng	0.4.1	reticulate	1.46.0
dtplyr	1.3.3	rhdf5	2.54.1
dynamicTreeCut	1.63-1	rhdf5filters	1.22.0
e1071	1.7-17	Rhdf5lib	1.32.0
ellipsis	0.3.3	RhpcBLASctl	0.23-42
enrichplot	1.30.5	Rhtslib	3.6.0
enrichR	3.4	rjson	0.2.23
eulerr	8.0.0	rlang	1.2.0
evaluate	1.0.5	rmarkdown	2.31
fansi	1.0.7	ROCR	1.0-12
farver	2.1.2	roxygen2	8.0.0
fastcluster	1.3.0	rpart	4.1.24
fastDummies	1.7.6	rprojroot	2.1.1
fastmap	1.2.0	rsample	1.3.2
fastmatch	1.1-8	Rsamtools	2.26.0
fgsea	1.36.2	RSpectra	0.16-2
fields	17.3	RSQLite	3.53.1
filelock	1.0.3	rstatix	0.7.3
fitdistrplus	1.2-6	rstudioapi	0.18.0
FNN	1.1.4.1	rsvd	1.0.5
fontawesome	0.5.3	Rtsne	0.17
fontBitstreamVera	0.1.1	rversions	3.0.0
fontLiberation	0.1.0	rvest	1.0.5
fontquiver	0.2.1	s2	1.1.11
forcats	1.0.1	S4Arrays	1.10.1
foreach	1.5.2	S4Vectors	0.48.1
forecast	9.0.2	S7	0.2.2
foreign	0.8-91	sass	0.4.10
formatR	1.14	ScaledMatrix	1.18.0
Formula	1.2-5	scales	1.4.0
fracdiff	1.5-4	scater	1.38.1
fs	2.1.0	scattermore	1.2
furrr	0.4.0	scatterpie	0.2.6
futile.logger	1.4.9	sctransform	0.4.3
futile.options	1.0.1	scuttle	1.20.0
future	1.70.0	selectr	0.5-1
future.apply	1.20.2	Seqinfo	1.0.0
gargle	1.6.1	sessioninfo	1.2.3
gdtools	0.5.1	Seurat	5.5.0
GeneOverlap	1.46.0	SeuratDisk	0.0.0.9021
generics	0.1.4	SeuratObject	5.4.0
GENIE3	1.32.0	SeuratWrappers	0.4.0
GenomeInfoDb	1.46.2	sf	1.1-1
GenomicRanges	1.62.1	shape	1.4.6.1
gert	2.3.1	shiny	1.13.0
GetoptLong	1.1.1	SingleCellExperiment	1.32.0
ggbeeswarm	0.7.3	sitmo	2.0.2

Package	Version	Package	Version
ggdist	3.3.3	slam	0.1-55
ggforce	0.5.0	slider	0.3.3
ggfun	0.2.0	snow	0.4-4
ggiraph	0.9.6	sourcetools	0.1.7-2
ggnewscale	0.5.2	sp	2.2-1
ggplot2	4.0.3	spam	2.11-4
ggplotify	0.1.3	SparseArray	1.10.10
ggpubr	0.6.3	SparseM	1.84-2
ggraph	2.2.2	sparseMatrixStats	1.22.0
ggrastr	1.0.2	spatial	7.3-18
ggrepel	0.9.8	spatstat.data	3.1-9
ggridges	0.5.7	spatstat.explore	3.8-1
ggsci	5.0.0	spatstat.geom	3.8-1
ggsignif	0.6.4	spatstat.random	3.5-0
ggtangle	0.1.2	spatstat.sparse	3.2-0
ggtree	4.0.5	spatstat.univar	3.2-0
ggvenn	0.1.19	spatstat.utils	3.2-3
gh	1.6.0	spData	2.3.5
gitcreds	0.1.2	spdep	1.4-2
GlobalOptions	0.1.4	speedglm	0.3-4
globals	0.19.1	statmod	1.5.2
glue	1.8.1	stringi	1.8.7
GO.db	3.22.0	stringr	1.6.0
goftest	1.2-3	SummarizedExperiment	1.40.0
googledrive	2.1.2	survival	3.8-6
googlesheets4	1.1.2	svglite	2.2.2
GOSemSim	2.36.0	sys	3.4.3
gplots	3.3.0	systemfonts	1.3.2
graphlayouts	1.2.3	tensor	1.5.1
gridExtra	2.3	tester	0.3.0
gridGraphics	0.5-1	testthat	3.3.2
grr	0.9.5	textshaping	1.0.5
gson	0.1.0	tibble	3.3.1
gtable	0.3.6	tidydr	0.0.6
gtools	3.9.5	tidygraph	1.3.1
h5mread	1.2.1	tidyr	1.3.2
harmony	2.0.3	tidyselect	1.2.1
haven	2.5.5	tidytree	0.4.7
HDF5Array	1.38.0	tidyverse	2.0.0
hdf5r	1.3.12	timechange	0.4.0
hdWGCNA	0.4.11	timeDate	4052.112
here	1.0.2	tinytex	0.59
hexbin	1.28.5	treeio	1.34.0
highr	0.12	tweenr	2.0.3
Hmisc	5.2-5	tzdb	0.5.0
hms	1.1.4	UCell	2.14.0
htmlTable	2.5.0	UCSC.utils	1.6.1
htmltools	0.5.9	units	1.0-1
htmlwidgets	1.6.4	UpSetR	1.4.1
httpuv	1.6.17	urca	1.3-4
httr	1.4.8	urlchecker	1.0.1
httr2	1.2.2	usethis	3.2.1
ica	1.0-3	utf8	1.2.6
ids	1.0.1	uuid	1.2-2
igraph	2.3.2	uwot	0.2.4
impute	1.84.0	vctrs	0.7.3
ini	0.3.1	VennDiagram	1.8.2
IRanges	2.44.0	vipor	0.4.7
irlba	2.3.7	viridis	0.6.5
isoband	0.3.0	viridisLite	0.4.3
iterators	1.0.14	vroom	1.7.1
jquerylib	0.1.4	waldo	0.6.2
jsonlite	2.0.0	warp	0.2.3
kableExtra	1.4.0	WGCNA	1.74
KEGGREST	1.50.0	whisker	0.4.1
KernSmooth	2.23-26	withr	3.0.2

Package	Version	Package	Version
knitr	1.51	wk	0.9.5
labeling	0.4.3	WriteXLS	6.8.0
lambda.r	1.2.4	xfun	0.58
later	1.4.8	xml2	1.5.2
lattice	0.22-9	xopen	1.0.1
lazyeval	0.2.3	xtable	1.8-8
leidenbase	0.1.37	XVector	0.50.0
lifecycle	1.0.5	yaml	2.3.12
limma	3.66.0	yulab.utils	0.2.4
listenv	0.10.1	zellkonverter	1.20.1
littler	0.3.23	zip	2.3.3
lme4	2.0-1	zoo	1.8-15
lmtest	0.9-40		

Python packages (207)

Package	Version	Package	Version
adjustText	1.3.0	notebook	6.5.7
anndata	0.12.16	notebook_shim	0.2.4
annotated-types	0.7.0	numba	0.65.1
anyio	4.13.0	numcodecs	0.16.5
argon2-cffi	25.1.0	numexpr	2.14.1
argon2-cffi-bindings	25.1.0	numpy	2.4.6
array-api-compat	1.14.0	numpy-groupies	0.11.3
arrow	1.4.0	nvidia-cublas	13.1.1.3
asttokens	3.0.1	nvidia-cuda-cupti	13.0.85
async-lru	2.3.0	nvidia-cuda-nvrtc	13.0.88
attrs	26.1.0	nvidia-cuda-runtime	13.0.96
babel	2.18.0	nvidia-cudnn-cu13	9.20.0.48
beautifulsoup4	4.14.3	nvidia-cufft	12.0.0.61
bleach	6.3.0	nvidia-cufile	1.15.1.6
blosc2	4.4.1	nvidia-curand	10.4.0.35
cellbender	0.3.2	nvidia-cusolver	12.0.4.66
certifi	2026.5.20	nvidia-cusparse	12.6.3.3
cffi	2.0.0	nvidia-cusparselt-cu13	0.8.1
charset-normalizer	3.4.7	nvidia-nccl-cu13	2.29.7
click	8.4.1	nvidia-nvjitlink	13.0.88
comm	0.2.3	nvidia-nvshmem-cu13	3.4.5
contourpy	1.3.3	nvidia-nvtx	13.0.85
cuda-bindings	13.3.1	opt_einsum	3.4.0
cuda-pathfinder	1.5.5	packaging	26.2
cuda-toolkit	13.0.2	palantir	1.4.4
cycler	0.12.1	pandas	2.3.3
debugpy	1.8.21	pandocfilters	1.5.1
decorator	5.3.1	parso	0.8.7
defusedxml	0.7.1	patsy	1.0.2
donfig	0.8.1.post1	pexpect	4.9.0
elpigraph-python	0.3.4	pillow	12.2.0
entrypoints	0.4	pip	26.1.2
executing	2.2.1	platformdirs	4.10.0
fa2_modified	0.4	plotly	6.7.0
fast-array-utils	1.4.1	prometheus_client	0.25.0
fastjsonschema	2.21.2	prompt_toolkit	3.0.52
filelock	3.29.0	psutil	7.2.2
fonttools	4.63.0	ptyprocess	0.7.0
fqdn	1.5.1	pure_eval	0.2.3
fsspec	2026.4.0	py-cpuinfo	9.0.0
google-crc32c	1.8.0	pycparser	3.0
h11	0.16.0	pydantic	2.13.4
h5py	3.16.0	pydantic_core	2.46.4
httpcore	1.0.9	Pygments	2.20.0
httpx	0.28.1	pynndescent	0.6.0
idna	3.18	pyparsing	3.3.2

Package	Version	Package	Version
igraph	1.0.0	pyro-api	0.1.2
ipykernel	6.29.5	pyro-ppl	1.9.1
ipython	9.14.0	python-dateutil	2.9.0.post0
ipython-genutils	0.2.0	python-igraph	1.0.0
ipython_pygments_lexers	1.1.1	python-json-logger	4.1.0
ipywidgets	8.1.8	pytz	2026.2
isoduration	20.11.0	PyYAML	6.0.3
jax	0.10.1	pyzmq	27.1.0
jaxlib	0.10.1	referencing	0.37.0
jaxopt	0.8.5	requests	2.34.2
jedi	0.20.0	rfc3339-validator	0.1.4
Jinja2	3.1.6	rfc3986-validator	0.1.1
joblib	1.5.3	rfc3987-syntax	1.1.0
json5	0.14.0	rpds-py	2026.5.1
jsonpointer	3.1.1	rpy2	3.6.7
jsonschema	4.26.0	rpy2-rinterface	3.6.6
jsonschema-specifications	2025.9.1	rpy2-robjects	3.6.5
jupyter	1.1.1	scanpy	1.12.1
jupyter-console	6.6.3	scFates	1.2.4
jupyter-events	0.12.1	scikit-learn	1.9.0
jupyter-highlight-selected-word	0.2.0	scikit-misc	0.5.2
jupyter-lsp	2.3.1	scipy	1.17.1
jupyter_client	7.4.9	scverse-misc	0.0.7
jupyter_contrib_core	0.4.2	seaborn	0.13.2
jupyter_contrib_nbextensions	0.7.0	Send2Trash	2.1.0
jupyter_core	5.9.1	session-info2	0.4.1
jupyter_nbextensions_configurator	0.6.4	setuptools	81.0.0
jupyter_server	2.19.0	shapely	2.1.2
jupyter_server_terminals	0.5.4	simpleppt	1.1.7
jupyterlab	4.5.7	six	1.17.0
jupyterlab_pygments	0.3.0	soupsieve	2.8.4
jupyterlab_server	2.28.0	stack-data	0.6.3
jupyterlab_widgets	3.0.16	statsmodels	0.14.6
kiwisolver	1.5.0	sympy	1.14.0
lark	1.3.1	tables	3.11.1
legacy-api-wrap	1.5	terminado	0.18.1
leidenalg	0.12.0	texttable	1.7.0
llvmlite	0.47.0	threadpoolctl	3.6.0
loompy	3.0.8	tinycss2	1.5.1
lxml	6.1.1	torch	2.12.0
lxml_html_clean	0.4.5	tornado	6.5.6
MarkupSafe	3.0.3	tqdm	4.67.3
matplotlib	3.10.9	traitlets	5.15.0
matplotlib-inline	0.2.2	triton	3.7.0
mellon	1.7.1	typing-inspection	0.4.2
mistune	0.8.4	typing_extensions	4.15.0
ml_dtypes	0.5.4	tzdata	2026.2
mpmath	1.3.0	tzlocal	5.4.4
msgpack	1.1.2	umap-learn	0.5.12
narwhals	2.22.0	uri-template	1.3.0
natsort	8.4.0	urllib3	2.7.0
nbclassic	1.3.3	wcwidth	0.7.0
nbclient	0.10.4	webcolors	25.10.0
nbconvert	6.5.4	webencodings	0.5.1
nbformat	5.10.4	websocket-client	1.9.0
ndindex	1.10.1	widgetsnbextension	4.0.15
nest-asyncio	1.6.0	zarr	3.2.1
networkx	3.6.1		